DEVSECOPS LAB BOOK • PHASE 10

Network Security

Chapter 30 • Firewalls, IDS/IPS, Segmentation & DDoS Defense

6-7 hrs	6	5
Time Estimate	Defense Layers	VLAN Segments

BY THE END OF THIS CHAPTER

Multi-layer firewalls • Suricata IDS/IPS • DDoS mitigation • VLAN segmentation • Threat intelligence

Edition 2026 Updated

Chapter Goal

Deploy advanced network security controls: multi-layer firewalls, intrusion detection and prevention systems (IDS/IPS), DDoS mitigation, network segmentation (VLANs), anomaly detection, and threat intelligence integration.

BY THE END OF THIS CHAPTER

- ✓ Multi-layer firewall architecture deployed
- ✓ Network Intrusion Detection System (Suricata) monitoring
- ✓ DDoS protection and rate limiting configured
- ✓ Network segmentation (VLANs) implemented
- ✓ Packet inspection and threat analysis
- ✓ Anomaly detection for network traffic
- ✓ DNS amplification protection
- ✓ Advanced firewall rules and state inspection

6-7 hours

Time estimate

30.1 — Network Security Defense in Depth

1	**Perimeter Firewall**	Stateful filtering, VPN termination, upstream DDoS protection — Cisco ASA, FortiGate, Palo Alto
2	**Host Firewall**	Per-system rules, application-aware filtering — UFW, iptables
3	**Network Segmentation**	VLANs isolate traffic; gateway-only inter-VLAN routing; limits lateral movement
4	**IDS/IPS**	Signature & anomaly detection, active threat blocking — Suricata, Snort, Zeek
5	**Application Firewall (WAF)**	SQLi/XSS/CSRF protection, rate limiting — ModSecurity, AWS WAF, Cloudflare
6	**Service Mesh Policies**	Container-to-container control, mTLS enforcement — Istio

Result: multiple checkpoints — compromising one layer doesn't compromise all.

30.2 — UFW Host Firewall: Baseline Setup

DEFAULT POLICIES (WHITELIST APPROACH)

`ufw default deny incoming`

Deny by default

`ufw default allow outgoing`

Allow outbound (can be restricted)

`ufw default deny routed`

Deny forwarded traffic

ESSENTIAL SERVICE RULES

```
ufw allow 22/tcp   # SSH
ufw allow 6443/tcp  # K8s API
ufw allow 80,443/tcp # HTTP/S
ufw allow from 10.0.0.0/8 to any port 5432  # Postgres
ufw limit 22/tcp   # brute-force rate limit
```

VERIFICATION & LOGGING

```
ufw status verbose
ufw show added
ufw show added numbered

ufw logging on
ufw logging medium

tail -f /var/log/ufw.log
grep UFW /var/log/syslog | tail -20
```

Rate limiting protects SSH against brute-force: ufw limit allows only 6 connection attempts per 30 seconds from the same IP before temporarily blocking it.

Advanced UFW Rules

Allow (Whitelist)

```
allow from 192.168.1.100 to any port 3306

allow from 10.0.0.0/8 to any port 5432

allow from 203.0.113.45 to any port 8443
```

Deny / Block

```
deny from 192.168.1.50

deny from 192.168.0.0/24

deny in on eth0 from 203.0.113.1
```

Ranges & Interfaces

```
allow 5000:6000/tcp

allow in on eth0 to 192.168.1.1 port 443

allow out on eth1 to 192.168.2.0/24 port 53
```

Rate Limit & Log

```
limit 22/tcp log

allow 443/tcp log

deny 23/tcp log-all
```

RULE MANAGEMENT

`ufw show numbered` • `ufw delete allow 80/tcp` • `ufw delete 5` • `ufw insert 1 allow from 10.0.0.0/8` • `ufw reload`

Caution: ufw reset clears all rules — use only in a controlled maintenance window.

30.3 — Network Segmentation: VLAN Design

VLAN	Purpose	Subnet	Contents
100	Management	`10.100.0.0/24`	Hypervisor mgmt, monitoring — admin-only access
200	K8s Control Plane	`10.200.0.0/24`	3 CP nodes, etcd, API server
300	K8s Workers	`10.300.0.0/24`	Worker nodes, container workloads, pod-to-pod
400	Databases	`10.400.0.0/24`	PostgreSQL, Redis, Elasticsearch — apps only
500	Applications/Users	`10.500.0.0/24`	User connections, ingress, load balancers

Design principle: traffic between VLANs only crosses through the gateway — never directly. This prevents lateral movement and reduces the blast radius of any single breach.

VLAN Traffic Flows & Benefits

ALLOWED

- ✓ Management → All (admin access)
- ✓ CP → Workers (orchestration)
- ✓ Workers ↔ Workers (pod-to-pod)
- ✓ Workers → Database (app queries)
- ✓ Apps → Workers (ingress)
- ✓ Internet → Apps (user traffic)

BLOCKED

- ✗ Workers → CP (no worker-initiated)
- ✗ Database → Workers (not db-initiated)
- ✗ Apps → Database (only through workers)
- ✗ User → Management (strict isolation)
- ✗ User → Database (no direct access)

BENEFITS

- ✓ Lateral movement limited to same VLAN
- ✓ Compromised app cannot reach database directly
- ✓ Compromised worker cannot reach control plane
- ✓ Administrative traffic isolated
- ✓ Blast radius of breaches reduced

30.4 — Intrusion Detection with Suricata

Suricata inspects network traffic in real time against signature and anomaly rules — running in IDS (detect/alert) or inline IPS (detect/block) mode.

KEY CONFIGURATION

HOME_NET	10.0.0.0/8 — internal address space
HTTP_PORTS	80, 8080, 8000, 8888, ... (web traffic)
rule-files	suricata.rules, custom.rules
eve-log outputs	alert, http, dns, tls, flow, stats (JSON)
threat-detection	TLS ja3-fingerprinting, SSH, HTTP inspection
threading	CPU affinity across decode/stream/detect stages

OPERATIONS

```
systemctl enable suricata
systemctl restart suricata

tail -f /var/log/suricata/eve.json \
  | jq 'select(.event_type=="alert")'

suricatasc -c "stats" \
  /var/run/suricata/suricata-command.socket

0 3 * * * suricata-update && \
  systemctl reload suricata
```

Suricata Detection Rules — Examples

SQL Injection	`alert http ... content:"SELECT"; content:"UNION";`	*Flags SELECT + UNION patterns in HTTP URI*
Malicious DNS Query	`alert dns ... dns.query; content:".ru";`	*Flags queries to known malicious TLDs/domains*
SSH Brute Force	`alert ssh ... byte_test:1,>,20,0,relative;`	*Flags abnormally high failed-auth byte patterns*
Buffer Overflow	`alert tcp ... byte_test:4,>,0,0;`	*Flags anomalous byte-length patterns in payload*
Suspicious TLS Cert	`alert tls ... tls.cert_subject; content:"*";`	*Flags certificates matching suspicious subject patterns*

30.5 — DDoS Mitigation Strategies

Kernel / iptables

- `tcp_syncookies = 1`
- `tcp_max_syn_backlog = 2048`
- `netdev_max_backlog = 5000`
- `tcp_abort_on_overflow = 1`

UFW Rate Limiting

- `ufw limit 22/tcp`
- `ufw limit 80/tcp`
- `ufw limit 443/tcp`

Fail2ban

- `bantime = 3600s`
- `maxretry = 5`
- `jails: sshd, nginx-http-auth,`
- `nginx-limit-req, http-get-dos`

HAProxy / Nginx

- `stick-table + http_req_rate`
- `limit_req_zone (10r/s, burst 20)`
- `limit_conn (10 per IP)`
- `client_body/header_timeout`

Layered rate limiting: kernel-level SYN cookies stop connection floods, UFW/iptables cap raw packet rates, fail2ban blocks repeat offenders, and HAProxy/Nginx enforce per-IP request quotas at the application layer.

DDoS Detection & Monitoring

```
watch -n 1 'netstat -an | grep ESTABLISHED | wc -l'
```

Monitor live connection count

```
netstat -an | grep ESTABLISHED | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn
```

Top connections per source IP

```
netstat -an | grep SYN | wc -l
```

Detect SYN flood volume

```
tcpdump -i eth0 -c 1000 'tcp[tcpflags] & tcp-syn != 0'
```

Capture & rank suspicious SYN traffic

```
tail -f eve.json | jq 'select(.flow.pkts_toserver > 1000)'
```

Suricata: flag high-volume flows

30.6 — Chapter Summary

Security Layer	Tools	Purpose
Perimeter	UFW, iptables	Host-based firewall
Segmentation	VLANs, routing	Network isolation
Detection	Suricata, Snort	IDS/IPS
DDoS Protection	fail2ban, rate limiting	Attack mitigation
Logging	ELK, Splunk	Traffic analysis

Key Takeaway — Layered Network Defense: *network security requires multiple overlapping defenses. No single tool solves everything; defense in depth is essential.*

Chapter Complete

- ✓ Multi-layer firewall architecture deployed (perimeter → host → segmentation → IDS/IPS → WAF → service mesh)
- ✓ Suricata IDS/IPS actively monitoring and alerting on threats
- ✓ 5 VLANs segment management, control plane, workers, database, and app traffic
- ✓ DDoS mitigation layered from kernel to application (SYN cookies → fail2ban → rate limiting)
- ✓ Chapter 31 continues with centralized logging and SIEM integration

6	5	6-7 hrs
Defense Layers	VLAN Segments	Time Invested